



Evaluación Semana 4

Despliegue, contenedores e infraestructura inicial
del proyecto real

Grupo 2

Jonathan Roberto Acosta Lopez

Rene Alexander Araujo Soto

Mario Alexander Hernández Quevedo

Módulo 4

Desarrollo de Aplicaciones con IA Julio 2026

Descripción del servicio y ruta elegida

Descripción breve del servicio preparado

EduIA es una plataforma educativa con Inteligencia Artificial que permite a docentes generar evaluaciones automáticamente y a estudiantes recibir retroalimentación personalizada de código, el servicio preparado para despliegue es la API REST construida con FastAPI y Uvicorn, que expone las capacidades de IA del proyecto: evaluación de código, generación de cursos y chat educativo.

Ruta elegida: Docker local

Se eligió la ruta de contenedor Docker local, la API FastAPI se containeriza de forma independiente al frontend Streamlit ya que es el componente más portátil y sin estado del proyecto, el frontend Streamlit se mantiene en ejecución local por su dependencia con SQLite y la sesión de usuario en memoria.

Dockerfile — Configuración del contenedor de la API

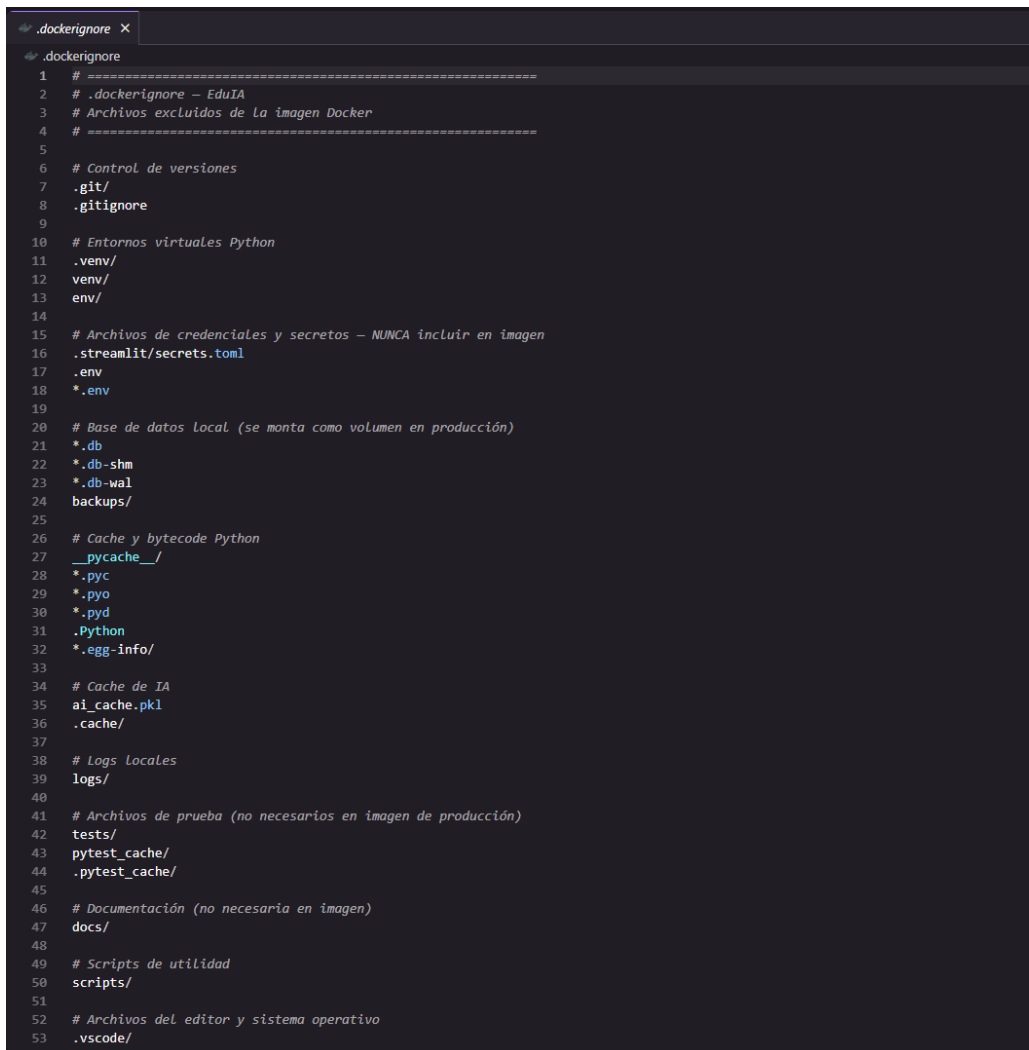
El Dockerfile utiliza la imagen base oficial python:3.11-slim para mantener el tamaño reducido instala las dependencias desde requirements-test.txt copia solo los archivos necesarios para la API e incluye un HEALTHCHECK que verifica el endpoint /health cada 30 segundos.

```
1 # -----
2 # EduIA - Dockerfile
3 # Imagen para la API FastAPI (servicio de IA)
4 # -----
5
6 # Imagen base oficial Python 3.11 slim
7 FROM python:3.11-slim
8
9 # Metadatos
10 LABEL maintainer="Grupo 2 - EduIA"
11 LABEL description="API REST inteligente de EduIA con FastAPI y Google Gemini"
12 LABEL version="1.0.0"
13
14 # Variables de entorno del contenedor
15 ENV PYTHONUNBUFFERED=1 \
16     PYTHONPATH=/app \
17     PORT=8000
18
19 # Directorio de trabajo dentro del contenedor
20 WORKDIR /app
21
22 # Copiar solo requirements primero (optimiza cache de capas)
23 COPY requirements-test.txt .
24
25 # Instalar dependencias
26 RUN pip install --no-cache-dir --upgrade pip && \
27     pip install --no-cache-dir -r requirements-test.txt
28
29 # Copiar el código fuente necesario para la API
30 COPY api.py .
31 COPY config.py .
32 COPY database.py .
33 COPY utils_ai.py .
34 COPY models/ ./models/
35 COPY evaluacion/ ./evaluacion/
36
37 # Crear directorio para la base de datos SQLite
38 RUN mkdir -p /app/data
39
40 # Puerto expuesto
41 EXPOSE 8000
42
43 # Healthcheck: verifica que la API responde
44 HEALTHCHECK --interval=30s --timeout=10s --start-period=15s --retries=3 \
45     CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health') || exit 1"
46
47 # Comando de inicio
48 CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
49 |
```

Dockerignore

El archivo `.dockerignore` excluye de la imagen Docker los siguientes elementos para mantener la imagen limpia, segura y liviana:

- `.streamlit/secrets.toml` — credenciales y API keys (nunca deben entrar a la imagen)
- `*.db`, `backups/` — base de datos local (se monta como volumen en producción)
- `tests/`, `docs/`, `scripts/` — archivos de desarrollo no necesarios en producción
- `__pycache__/, *.pyc` — bytecode de Python
- `ai_cache.pkl`, `logs/` — archivos generados en tiempo de ejecución
- `.git/`, `.github/` — historial de versiones y CI/CD

A screenshot of a code editor showing the content of a `.dockerignore` file. The file is titled `.dockerignore` in the tab. The content is a list of patterns to ignore, with comments in Spanish explaining each section. The patterns include `.git/`, `.gitignore`, `.venv/`, `venv/`, `env/`, `.streamlit/secrets.toml`, `.env`, `*.env`, `*.db`, `*.db-shm`, `*.db-wal`, `backups/`, `__pycache__/`, `*.pyc`, `*.pyo`, `*.pyd`, `.Python`, `*.egg-info/`, `ai_cache.pkl`, `.cache/`, `logs/`, `tests/`, `pytest_cache/`, `.pytest_cache/`, `docs/`, `scripts/`, and `.vscode/`.

```
.dockerignore
# =====
# .dockerignore - EduIA
# Archivos excluidos de la imagen Docker
# =====
# Control de versiones
.git/
.gitignore
# Entornos virtuales Python
.venv/
venv/
env/
# Archivos de credenciales y secretos - NUNCA incluir en imagen
.streamlit/secrets.toml
.env
*.env
# Base de datos Local (se monta como volumen en producción)
*.db
*.db-shm
*.db-wal
backups/
# Cache y bytecode Python
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
*.egg-info/
# Cache de IA
ai_cache.pkl
.cache/
# Logs locales
logs/
# Archivos de prueba (no necesarios en imagen de producción)
tests/
pytest_cache/
.pytest_cache/
# Documentación (no necesaria en imagen)
docs/
# Scripts de utilidad
scripts/
# Archivos del editor y sistema operativo
.vscode/
```

Variables de entorno requeridas

Las variables de entorno se documentan en el archivo .env.example del repositorio ninguna clave real está expuesta en el código fuente ni en la imagen Docker.

Variable	Obligatoria	Descripción	Cómo configurar
GEMINI_API_KEY	Sí (para IA)	Clave de Google AI Studio para Gemini	-e GEMINI_API_KEY=tu_key en docker run
TESTING	No	Activa modo test sin Gemini real	-e TESTING=true
DEBUG	No	Modo debug de la aplicación	-e DEBUG=false

Comando completo con variables:

docker run -d -p 8000:8000 -e GEMINI_API_KEY=tu_key -e TESTING=false --name eduia-api eduia-api:1.0

Evidencia 1 — Construcción de la imagen Docker

Se ejecutó el comando docker build -t eduia-api:1.0 . desde la raíz del proyecto, el proceso completó exitosamente las 11 capas definidas en el Dockerfile: descarga de imagen base Python 3.11-slim, instalación de dependencias, copia de archivos fuente y creación del directorio de datos. Resultado: imagen eduia-api:1.0 lista en el registro local de Docker.

```
C:\Users\jonar\Downloads\Nueva carpeta (6)\Proyecto-eduIA>docker build -t eduia-api:1.0 .
[+] Building 1.2s (17/17) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 1.47kB                             0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.0s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 1.23kB                                   0.0s
=> [ 1/11] FROM docker.io/library/python:3.11-slim@sha256:db3ff2e1800a8581e2c48a27c3995339d47bdf046da21c7627accd 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:db3ff2e1800a8581e2c48a27c3995339d47bdf046da21c7627accd3d 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 1.77kB                                   0.0s
=> CACHED [ 2/11] WORKDIR /app                                     0.0s
=> CACHED [ 3/11] COPY requirements-test.txt .                    0.0s
=> CACHED [ 4/11] RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements 0.0s
=> CACHED [ 5/11] COPY api.py .                                    0.0s
=> CACHED [ 6/11] COPY config.py .                                0.0s
=> CACHED [ 7/11] COPY database.py .                              0.0s
=> CACHED [ 8/11] COPY utils_ai.py .                              0.0s
=> CACHED [ 9/11] COPY models/ ./models/                         0.0s
=> CACHED [10/11] COPY evaluacion/ ./evaluacion/                 0.0s
=> CACHED [11/11] RUN mkdir -p /app/data                         0.0s
=> exporting to image                                             0.1s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:e963819d796dcb619582169a00ed8c871e1c72b7d3553b08f26426f0b48ff1c0 0.0s
=> => exporting config sha256:de45e092daf6c7132fa9863c0efd21dd25b295878295d7f7ccd3f2bb73929089e 0.0s
=> => exporting attestation manifest sha256:48ccd4cb0ab2c608f40789d9bdd7e957a7959cab98bac502e23f6bcd5771d974 0.0s
=> => exporting manifest list sha256:592ac3934518b7031f684abb436f45d5a2966e4954561fc8e0847e7933a033ff 0.0s
=> => naming to docker.io/library/eduia-api:1.0                  0.0s
=> => unpacking to docker.io/library/eduia-api:1.0               0.0s
C:\Users\jonar\Downloads\Nueva carpeta (6)\Proyecto-eduIA>
```

Evidencia 2 — Ejecución del contenedor

Se ejecutó docker run con el puerto 8000 mapeado y las variables de entorno configuradas, el mensaje de conflicto de nombre indica que el contenedor ya estaba corriendo de una ejecución anterior, el comando docker ps confirma el contenedor eduia-api activo con estado Up 9 minutes (healthy) y el puerto 0.0.0.0:8000->8000/tcp correctamente expuesto.

```
C:\Users\jonar\Downloads\Nueva carpeta (6)\Proyecto-eduIA>docker run -d -p 8000:8000 -e GEMINI_API_KEY="test-key-demo" -e TESTING="true" --name eduia-api eduia-api:1.0

What's next:
  Debug this container error with Gordon + docker ai "help me fix this container error"
docker: Error response from daemon: Conflict. The container name "/eduia-api" is already in use by container "b999425678eb486dc4396274cd1647e54fcadd44c03155a068d240f98fc963f". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information

C:\Users\jonar\Downloads\Nueva carpeta (6)\Proyecto-eduIA>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS                               NAMES
b999425678eb   c7b19e820904   "uvicorn api:app --h..." 9 minutes ago        Up 9 minutes (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  eduia-api

C:\Users\jonar\Downloads\Nueva carpeta (6)\Proyecto-eduIA>
```

Evidencia 3 — Logs del servidor en ejecución

Los logs del contenedor confirman el inicio correcto del servidor se observa Application startup complete y Uvicorn running on <http://0.0.0.0:8000> el healthcheck interno ejecuta peticiones GET /health repetidas que responden 200 OK, se registra también una llamada exitosa a POST /api/evaluate 200 OK.

Se presentan dos advertencias esperadas y documentadas:

- **FutureWarning** de google.generativeai: advertencia de deprecación, no afecta el funcionamiento. Pendiente de migración en Semana 5.
- **No module named 'streamlit'**: esperado, el contenedor solo ejecuta la API FastAPI, no el frontend.

```
C:\Users\jonar\Downloads\Nueva carpeta (6)\Proyecto-eduIA>docker logs eduia-api
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:35350 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:47984 - "GET /health HTTP/1.1" 200 OK
INFO: 172.17.0.1:41272 - "GET /health HTTP/1.1" 200 OK
INFO: 172.17.0.1:48216 - "GET /health HTTP/1.1" 200 OK
2026-07-31 01:24:52,686 [INFO] eduia.api: evaluate_code - language=python len=32
/app/utlis_ai.py:8: FutureWarning:

All support for the 'google.generativeai' package has ended. It will no longer be receiving updates or bug fixes. Please switch to the 'google.genai' package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

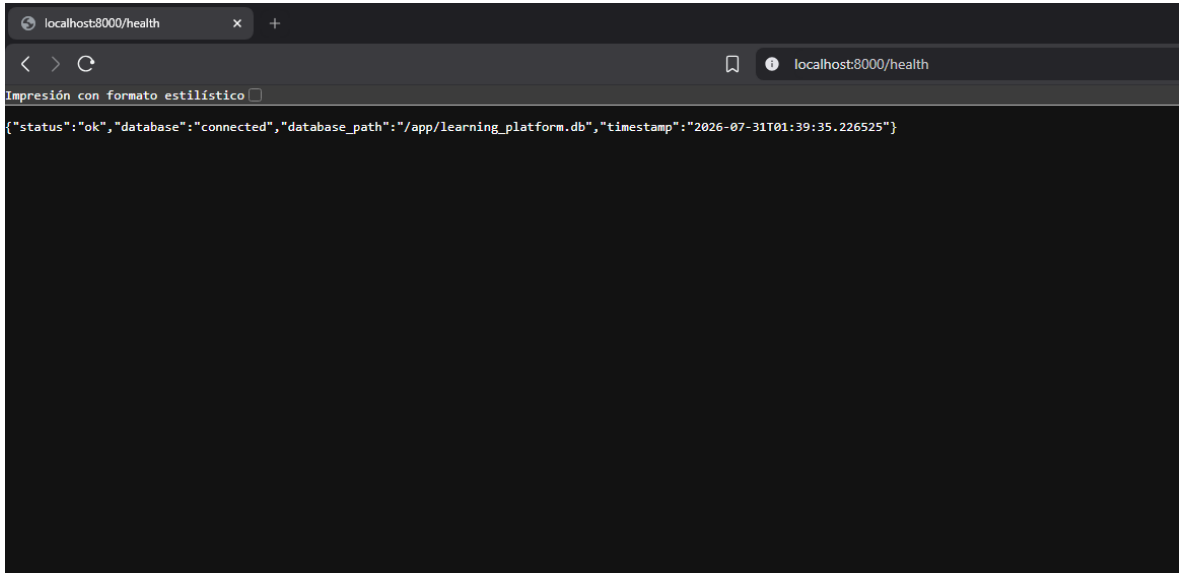
import google.generativeai as genai
2026-07-31 01:24:52,928 [ERROR] eduia.api: Error instanciando AIManager: No module named 'streamlit'
INFO: 172.17.0.1:48216 - "POST /api/evaluate HTTP/1.1" 200 OK
INFO: 127.0.0.1:57288 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:37186 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:50196 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:34262 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:54410 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:39914 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:44030 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:50504 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:33056 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:50662 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:55674 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:41312 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:37420 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:59588 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:53422 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:55444 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:48962 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:36328 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:45270 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:37126 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:50788 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:57170 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:32936 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:48542 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:37076 - "GET /health HTTP/1.1" 200 OK

C:\Users\jonar\Downloads\Nueva carpeta (6)\Proyecto-eduIA>
```

Evidencia 4 — Prueba del endpoint /health

Se accedió desde el navegador a <http://localhost:8000/health>. El endpoint retornó HTTP 200 con el siguiente JSON, confirmando que el servicio está operativo y la base de datos conectada:

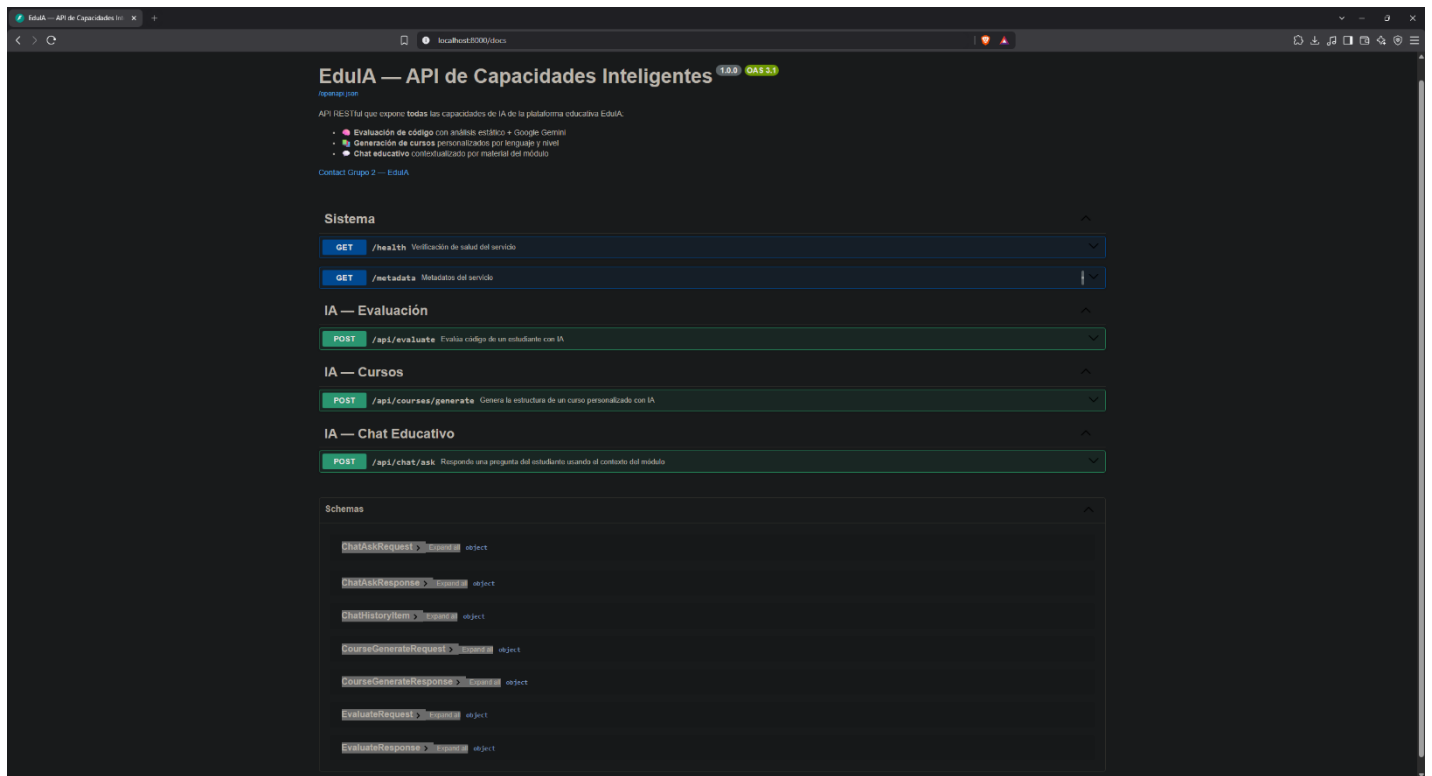
```
{"status":"ok","database":"connected","database_path":"/app/learning_platform.db","timestamp":"2026-07-31T01:39:35.226525"}
```



Evidencia 5 — Endpoint principal: Swagger UI (/docs)

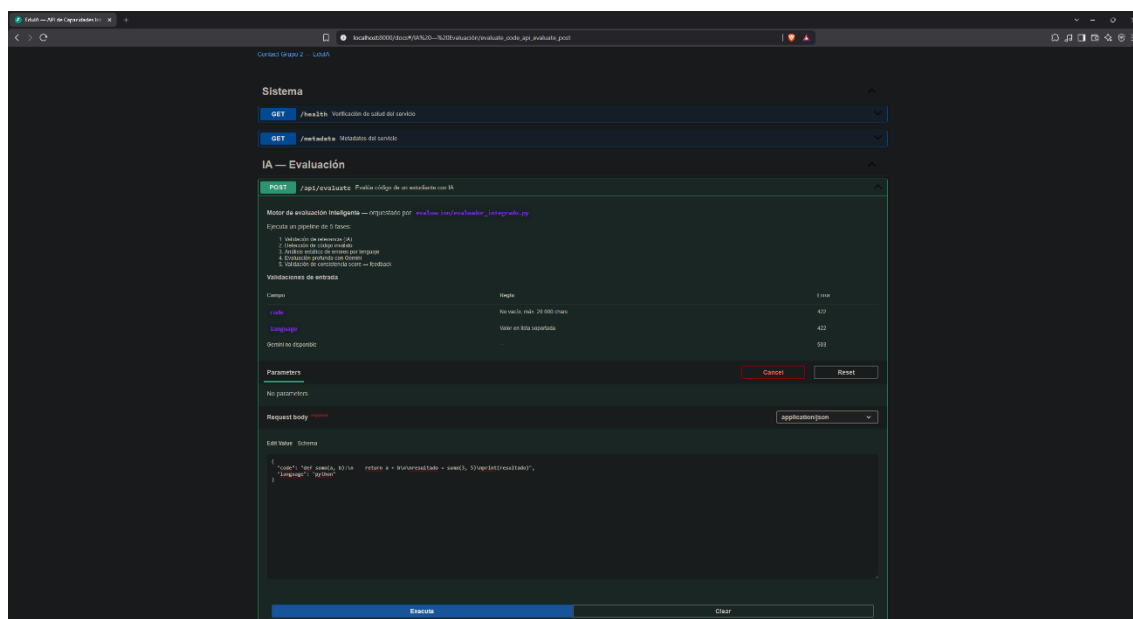
Se accedió a <http://localhost:8000/docs> que muestra la documentación interactiva generada automáticamente por FastAPI se confirma la disponibilidad de los endpoints principales del proyecto:

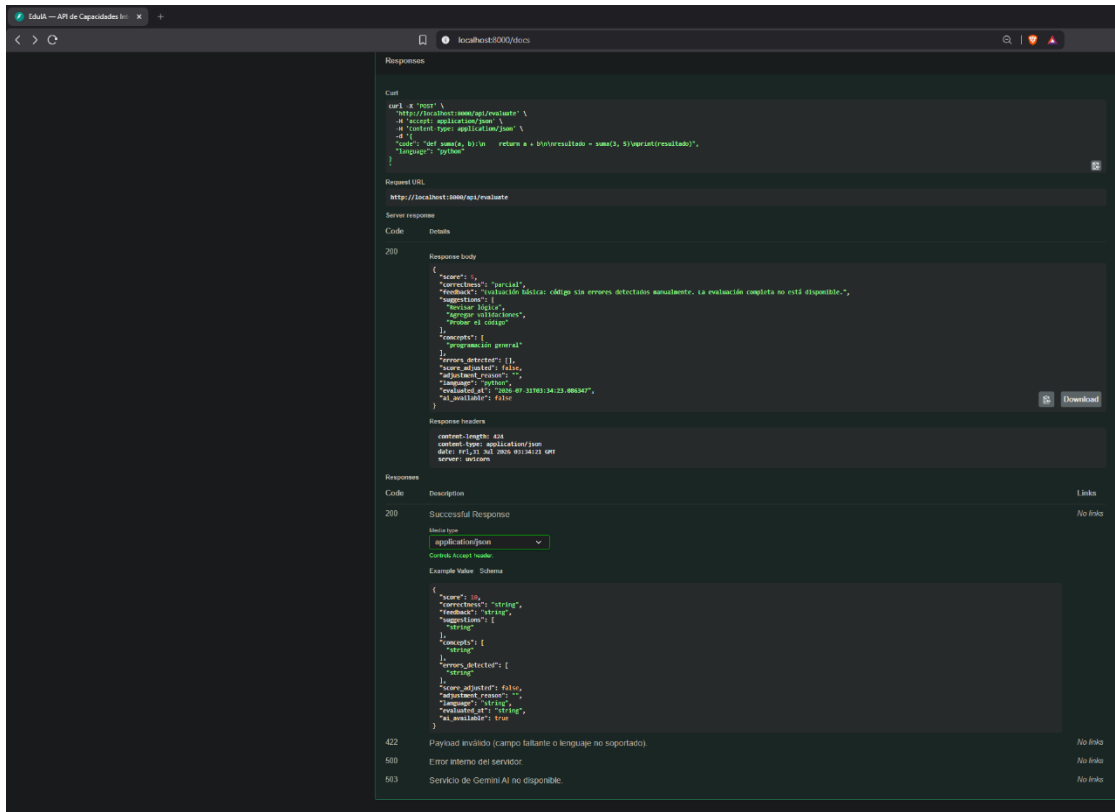
- GET /health — Verificación de salud del servicio
- GET /metadata — Metadatos del servicio
- POST /api/evaluate — Evaluación de código con IA (Google Gemini)
- POST /api/courses/generate — Generación de cursos personalizados con IA
- POST /api/chat/ask — Chat educativo contextualizado por material del módulo



Evidencia 5B — Ejecución real del endpoint POST /api/evaluate

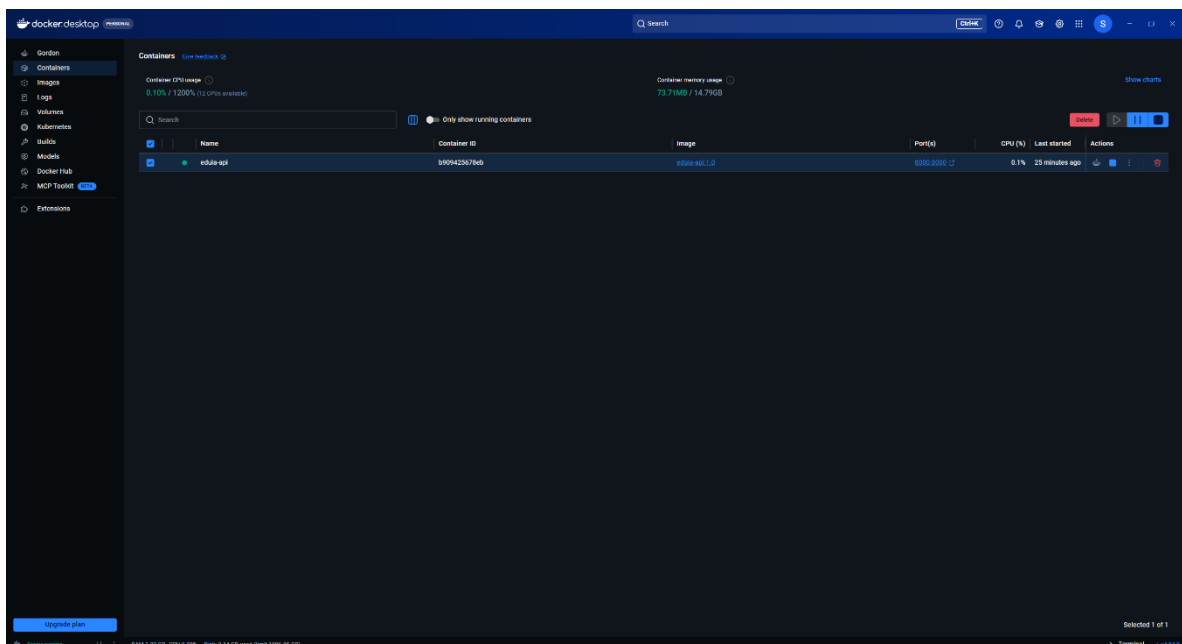
Se ejecutó una petición real al endpoint POST /api/evaluate desde Swagger UI, enviando código Python con una función suma el servidor respondió con HTTP 200 y retornó correctamente la respuesta del servicio, incluyendo score, feedback y sugerencias ,esto confirma que el endpoint principal procesa peticiones correctamente desde el contenedor Docker.





Evidencia 6 — Vista desde Docker Desktop

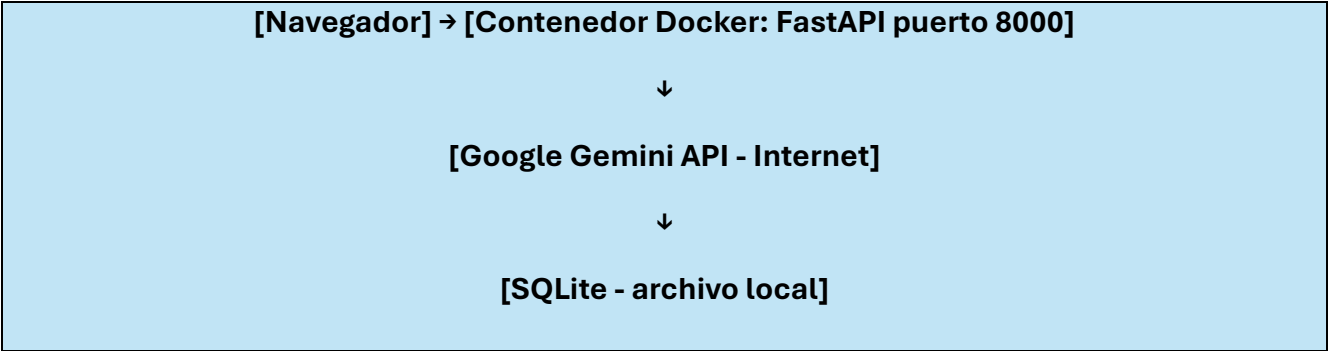
Docker Desktop confirma el contenedor eduia-api corriendo con la imagen eduia-api:1.0, puerto 8000 activo hace 25 minutos. El uso de CPU es mínimo (0.1%) indicando que el servicio está en espera de peticiones de forma eficiente.



Errores, intentos, bloqueos y correcciones

#	Error	Causa	Solución
1	Conflict: container name already in use	El contenedor ya estaba corriendo de una ejecución previa	Se usa docker stop eduia-api && docker rm eduia-api antes de volver a ejecutar
2	FutureWarning: google.generativeai	Librería google-generativeai deprecada por Google	Advertencia no crítica, pendiente migrar a google.genai en Semana 5
3	No module named 'streamlit'	Streamlit no está instalado en el contenedor de la API	Comportamiento esperado y correcto. El contenedor solo ejecuta FastAPI

Plan de infraestructura mínima



Componente	Tecnología	Puerto
API REST	FastAPI + Uvicorn (Docker)	8000
Frontend	Streamlit	8501
Base de datos	SQLite	—
IA	Google Gemini API	443 HTTPS

Estimación de costos iniciales

Servicio	Plan	Costo/mes	Notas
Google Gemini API	Free tier (1,500 req/día)	\$0	Google AI Studio, sin tarjeta de crédito
Docker Desktop	Personal	\$0	Gratis para uso personal y educativo
Render (PaaS despliegue)	Free tier	\$0	Despliega contenedores gratis, se duerme tras 15 min de inactividad
GitHub Actions (CI/CD)	Free tier (2,000 min/mes)	\$0	Ya integrado en el proyecto
GitHub (repositorio)	Free	\$0	Repositorio público gratuito
SQLite (base de datos)	Incluido en Python	\$0	No requiere servidor externo
Total actual (académico)	Solo free tiers	\$0 USD	Stack completamente gratuito
Total para producción real	Alternativas gratuitas	\$0 USD	Suficiente para prototipo académico

Supuestos del plan gratuito:

- Uso académico con menos de 100 usuarios simultáneos
- Menos de 1,500 llamadas diarias a Gemini (límite del free tier)
- El "sleep" de Render en inactividad es aceptable para un prototipo
- El repositorio es público en GitHub

Alternativa a Render si se necesita más disponibilidad:

- **Hugging Face Spaces** — despliega apps Python/Docker gratis sin sleep
- **Koyeb** — free tier con 1 servicio siempre activo, sin tarjeta de crédito
- **Fly.io** — free tier con 3 VMs pequeñas incluidas

Riesgos técnicos pendientes

#	Riesgo	Impacto	Plan
1	SQLite no soporta múltiples instancias del contenedor	Alto	Migrar a PostgreSQL en Semana 5-6
2	Librería google-generativeai deprecada	Medio	Migrar a google.genai en Semana 5
3	Sin HTTPS en el contenedor local	Medio	Agregar Nginx como reverse proxy en despliegue real
4	Sin autenticación en endpoints de la API	Alto	Agregar API Key o JWT en Semana 5
5	Sin rate limiting en la API	Medio	Agregar slowapi en Semana 5

Repositorio del proyecto

Repositorio GitHub: <https://github.com/Rene29Alexander/Proyecto-eduIA>

El repositorio contiene el código fuente actualizado, Dockerfile, .dockerignore, .env.example, pruebas automatizadas, pipeline CI/CD (.github/workflows/ci.yml), documentación técnica en /docs y el historial completo de cambios realizados durante el módulo.